CAMPUS RECRUITMENT TRAINING & PLACEMENT BOARD

OFFICIAL PRESCRIBED PLACEMENT MASTER TEXTBOOK

SQL & Relational Databases: Placement Master Edition (Prescribed Textbook)

Subject Track: SQL & Database Queries

Edition: 2026 Campus Placement Edition

SYLLABUS & CURRICULUM OVERVIEW

Complete enterprise textbook on SQL and relational database engineering covering relational algebra, normalization, query execution order, advanced window functions, indexing, transactions, and solved technical queries.

STRICT EXAMINATION GROUNDING GUARANTEE

All campus placement assessments, live code challenges, milestone tests, and company interview drills for SQL & Database Queries on this platform are derived strictly and exclusively from the theoretical concepts, algorithms, worked examples, and placement problems documented in this official textbook. Mastery of these chapters guarantees 100% exam preparation.

PRESCRIBED CHAPTER SYLLABUS (8 CHAPTERS):

Ch 1: Relational Theory, Schema Design & Normalization

Ch 2: SQL Query Execution Order & Deep Filtering Mechanics

Ch 3: Mastering SQL Joins & Set Operations

Ch 4: Subqueries, Correlated Subqueries & CTEs

Ch 5: Advanced Window Functions & Analytical SQL

Ch 6: Database Indexing, Query Plans & Performance Optimization

Ch 7: Transactions, ACID Properties & Concurrency Isolation

Ch 8: Solved Placement SQL Queries & Technical Coding Challenges

Author: Prescribed Courseware Architectural Board

Verified Complete Unabridged Placement Master Edition | Internal Campus Circulation

Edition: 2026 Campus Placement Edition

Subject: SQL & Database Queries

Courseware Board: Prescribed Placement Architectural Board

Status: In-App Complete Master Textbook (Full Curriculum Available In-Platform)

> **? EXAM GROUNDING POLICY:** All campus placement technical assessments, coding lab challenges, and technical interview evaluations are strictly grounded in the concepts, algorithms, formulas, step-by-step solved examples, practice questions, and interview questions detailed in this textbook.

· Course Overview & Curriculum Scope

Complete enterprise textbook on SQL and relational database engineering covering relational algebra, normalization, query execution order, advanced window functions, indexing, transactions, and solved technical queries.

· Complete Table of Contents (Chapter-by-Chapter)

- [Chapter 1: Relational Theory, Schema Design & Normalization](#chapter-1)
- [Chapter 2: SQL Query Execution Order & Deep Filtering Mechanics](#chapter-2)
- [Chapter 3: Mastering SQL Joins & Set Operations](#chapter-3)
- [Chapter 4: Subqueries, Correlated Subqueries & CTEs](#chapter-4)
- [Chapter 5: Advanced Window Functions & Analytical SQL](#chapter-5)
- [Chapter 6: Database Indexing, Query Plans & Performance Optimization](#chapter-6)
- [Chapter 7: Transactions, ACID Properties & Concurrency Isolation](#chapter-7)
- [Chapter 8: Solved Placement SQL Queries & Technical Coding Challenges](#chapter-8)

CHAPTER 1

Relational Theory, Schema Design & Normalization

Key Principle / Exam Takeaway: Master relational model principles, candidate keys, primary keys, foreign keys, functional dependencies, and lossless normal forms (1NF through BCNF).

Core Curriculum Topics Covered in this Chapter:

- Relational Model: Tuples, attributes, domains, and relational integrity constraints
- Super Keys, Candidate Keys, Primary Keys, and Foreign Key constraints
- Functional Dependencies (FDs) and Armstrong's Axioms
- Normalization hierarchy: First Normal Form (1NF), 2NF, 3NF, and Boyce-Codd (BCNF)
- Lossless-join decomposition and dependency preservation theorems

· 1. Core Concepts & In-Depth Explanation

Relational databases are built on Edgar F. Codd's Relational Model, which structures data into formal relations (tables) consisting of tuples (rows) and attributes (columns). Unlike hierarchical or flat-file structures, relations guarantee mathematical set semantics and enforce domain integrity.

Schema design directly impacts query performance and data anomalies:

- **Insertion Anomaly:** Inability to record certain facts without adding unrelated attributes.
- **Deletion Anomaly:** Unintended loss of crucial information when deleting related records.
- **Update Anomaly:** Redundant duplicate data causing data inconsistency when one record is updated and others are missed.

Normalization decomposes unnormalized tables into structured relations to eliminate redundancy without losing information. A decomposition is valid if and only if it is **Lossless-Join** (joining decomposed tables perfectly reconstructs the original relation without spurious tuples).

· 2. Key Formulas, Rules & Complexity Classes

1. **Armstrong's Axioms for Functional Dependencies:**
   - **Reflexivity:** If $Y \subseteq X$, then $X \to Y$.
   - **Augmentation:** If $X \to Y$, then $XZ \to YZ$.
   - **Transitivity:** If $X \to Y$ and $Y \to Z$, then $X \to Z$.
2. **Normal Form Definitions:**
   - **1NF:** All attribute values must be atomic (no multi-valued or composite attributes).
   - **2NF:** In 1NF and no non-prime attribute is partially dependent on any candidate key.
   - **3NF:** In 2NF and for every non-trivial FD $X \to Y$, either X is a super key or Y is a prime attribute.

- **BCNF:** For every non-trivial FD $X \to Y$, X must be a super key.

· 3. Step-by-Step Solved Examples

Example 1.1: Decomposing Schema to 3NF & BCNF

- **Difficulty / Target Companies:** `Medium | Oracle, Amazon, TCS Digital`
- **Problem Statement:** Given relation $R(A, B, C, D)$ with FDs: $A \to B, B \to C, C \to D$. Identify candidate keys, check current normal form, and decompose to BCNF.

Step-by-Step Walkthrough & Logic:

- Step 1: Compute closure of A: $A^+ = \{A, B, C, D\}$. A is the sole candidate key.
- Step 2: Check FDs against BCNF rule (determinant must be super key): In $B \to C$, B is not a super key ($B^+ = \{B, C, D\} \ne R$). In $C \to D$, C is not a super key. Thus R violates BCNF.
- Step 3: Check 3NF rule: In $B \to C$, C is not a prime attribute (only A is prime). Thus R also violates 3NF.
- Step 4: Decompose R based on violating FD $B \to C$: decompose into $R_1(B, C, D)$ and $R_2(A, B)$.
- Step 5: In R_1, $C \to D$ still violates BCNF. Decompose R_1 into $R_{11}(C, D)$ and $R_{12}(B, C)$.
- Step 6: Final BCNF relations: $R_a(A, B), R_b(B, C), R_c(C, D)$.

Complete Solved Code / Calculation:

```
-- Production Schema in BCNF
CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(100) NOT NULL
);

CREATE TABLE Employee (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(100) NOT NULL,
    dept_id INT NOT NULL,
    salary DECIMAL(10, 2) CHECK (salary > 0),
    CONSTRAINT fk_dept FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);
```

- **Line-by-Line Breakdown & Complexity Analysis:** Every table in the decomposed schema has a single candidate key as its primary key, eliminating all update and deletion anomalies while maintaining lossless joins.

· 4. Practice Questions with Answers & Explanations

Practice Question 1.1

Question: If a relation $R(A, B, C)$ has FD $AB \to C$ and $C \to B$, what is the highest normal form of R?

Options:

- A) 1NF
- B) 2NF

- C) 3NF
- D) BCNF

Answer: **C) 3NF** **Detailed Solution & Reasoning:** Candidate keys are AB and AC. In $C \to B$, C is not a super key, but B is a prime attribute (part of key AB). Thus it satisfies 3NF, but violates BCNF.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 1.1 (Amazon)

- **Interview Question:** _What is the difference between 3NF and BCNF, and why is 3NF sometimes preferred in real-world systems?_
- **Model High-Scoring Answer:** In 3NF, a functional dependency $X \to Y$ is allowed if X is a super key OR Y is a prime attribute. In BCNF, X MUST strictly be a super key. BCNF eliminates all redundancy based on functional dependencies, but decomposing into BCNF may lose dependency preservation. 3NF guarantees both lossless join and dependency preservation, which avoids expensive cross-table joins during integrity verification.

- **Interviewer Follow-Up & Pro Tip:** ? Clearly mention dependency preservation as the critical trade-off between 3NF and BCNF.

CHAPTER 2

SQL Query Execution Order & Deep Filtering Mechanics

Key Principle / Exam Takeaway: Master the physical logical execution order of SQL statements: FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> DISTINCT -> ORDER BY -> LIMIT.

Core Curriculum Topics Covered in this Chapter:

- Logical SQL execution lifecycle vs textual syntactic order
- WHERE vs HAVING: Row-level filtering vs aggregated group filtering
- NULL handling mechanics, three-valued logic (TRUE, FALSE, UNKNOWN), and COALESCE
- Pattern matching with LIKE, ILIKE, and regular expressions (REGEXP)
- CASE WHEN conditional expressions in projections and aggregations

· 1. Core Concepts & In-Depth Explanation

A common trap in campus placement technical interviews is confusing the written syntax order of SQL with its actual logical query processing order. While SQL queries begin with `SELECT`, the database query engine evaluates the query in this strict sequence: 1. `FROM` & `JOIN`: Identifies data sources and constructs the Cartesian product or joined virtual table. 2. `WHERE`: Filters individual rows before aggregation. Cannot reference aliases created in `SELECT`. 3. `GROUP BY`: Aggregates remaining rows into unique group buckets. 4. `HAVING`: Filters aggregated groups based on aggregate conditions (e.g., `COUNT(*) > 5`). 5. `SELECT`: Evaluates expressions, column projections, and assign column aliases. 6. `DISTINCT`: Removes duplicate rows from the projected result set. 7. `ORDER BY`: Sorts the final output. (Can reference aliases from `SELECT`). 8. `LIMIT` / `OFFSET`: Returns the specified page or slice of rows.

· 2. Key Formulas, Rules & Complexity Classes

1. **Logical Execution Order:** $$\text{FROM} \to \text{WHERE} \to \text{GROUP BY} \to \text{HAVING} \to \text{SELECT} \to \text{DISTINCT} \to \text{ORDER BY} \to \text{LIMIT}$$ 2. **Three-Valued Logic Truth Table:** $$\text{NULL} = \text{NULL} \implies \text{UNKNOWN (Evaluates to FALSE in WHERE clause)}$$ $$\text{Use: } \text{column IS NULL} \quad \text{or} \quad \text{COALESCE(column, default_value)}$$

· 3. Step-by-Step Solved Examples

Example 2.1: Filter Departments with High Average Salary

- **Difficulty / Target Companies:** `Easy | Infosys, TCS, Cognizant`
- **Problem Statement:** Write an SQL query to find departments with more than 3 employees where the average salary exceeds $75,000.

Step-by-Step Walkthrough & Logic:

- Step 1: Identify base table: `Employee`.
- Step 2: Group rows by `department_id`.
- Step 3: Filter groups using `HAVING` because both count and average salary require aggregation: `COUNT(*) > 3` AND `AVG(salary) > 75000`.
- Step 4: Select `department_id`, employee count, and average salary rounded to 2 decimals.

Complete Solved Code / Calculation:

```
SELECT
    department_id,
    COUNT(emp_id) AS total_employees,
    ROUND(AVG(salary), 2) AS avg_salary
FROM Employee
WHERE is_active = TRUE
GROUP BY department_id
HAVING COUNT(emp_id) > 3
   AND AVG(salary) > 75000.00
ORDER BY avg_salary DESC;
```

- **Line-by-Line Breakdown & Complexity Analysis:** WHERE filters inactive employees prior to grouping. GROUP BY clusters active employees by department. HAVING evaluates aggregated group metrics.

· 4. Practice Questions with Answers & Explanations

Practice Question 2.1

Question: Why will the query `SELECT name, salary * 1.1 AS new_sal FROM Emp WHERE new_sal > 50000;` fail?

Options:

- A) Arithmetic operations are disallowed in SELECT
- B) The WHERE clause executes before SELECT, so the alias `new_sal` does not yet exist
- C) Aliases cannot start with letters
- D) Emp table must be sorted

Answer: **B) The WHERE clause executes before SELECT, so the alias `new_sal` does not yet exist** **Detailed Solution & Reasoning:** Because WHERE executes before SELECT, column aliases defined in SELECT are not yet available to WHERE.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 2.1 (Microsoft)

- **Interview Question:** _What is the difference between COUNT(*), COUNT(1), and COUNT(column_name)?_
- **Model High-Scoring Answer:** Both `COUNT(*)` and `COUNT(1)` count the total number of rows matching the query criteria, including rows containing NULLs (modern query optimizers treat them identically). In contrast, `COUNT(column_name)` counts only rows where `column_name` is NOT NULL.
- **Interviewer Follow-Up & Pro Tip:** ? Highlight that `COUNT(column)` ignores NULL values.

CHAPTER 3

Mastering SQL Joins & Set Operations

Key Principle / Exam Takeaway: Master INNER, LEFT, RIGHT, FULL OUTER, CROSS, and SELF joins, plus UNION vs UNION ALL set mechanics.

Core Curriculum Topics Covered in this Chapter:

- Join mechanics: Cartesian product filtering vs hash joins vs merge joins
- INNER JOIN vs LEFT/RIGHT OUTER JOIN: Handling unmatched records
- FULL OUTER JOIN emulation in MySQL via UNION
- Self Join patterns for hierarchical parent-child organizational trees
- Set operators: UNION, UNION ALL, INTERSECT, and EXCEPT / MINUS

· 1. Core Concepts & In-Depth Explanation

Joins combine rows from two or more tables based on a related column between them:

- **INNER JOIN:** Returns only rows with matching values in both tables.
- **LEFT (OUTER) JOIN:** Returns all rows from the left table, and matched rows from the right table. Unmatched right table columns are populated with `NULL`.
- **RIGHT (OUTER) JOIN:** Returns all rows from the right table, and matched rows from the left table.
- **FULL (OUTER) JOIN:** Returns all rows when there is a match in either left or right table.
- **CROSS JOIN:** Cartesian product producing $M \times N$ rows.
- **SELF JOIN:** A table joined with itself, essential for hierarchical data (such as mapping Employees to their Managers).

Set operations combine outputs of multiple `SELECT` queries having identical column types. `UNION` performs an implicit `DISTINCT` deduplication sort pass, whereas `UNION ALL` preserves duplicates and executes significantly faster.

· 2. Key Formulas, Rules & Complexity Classes

1. **Join Cardinality Bounds:** For Table A with m rows and Table B with n rows: $$0 \le \text{Rows}(\text{INNER JOIN}) \le m \times n$$ $$m \le \text{Rows}(\text{LEFT JOIN}) \le m \times n$$ $$\text{Rows}(\text{CROSS JOIN}) = m \times n$$ 2. **UNION vs UNION ALL:** $$\text{Performance}(\text{UNION ALL}) \gg \text{Performance}(\text{UNION})$$

· 3. Step-by-Step Solved Examples

Example 3.1: Find Employees Earning More Than Their Managers (Self Join)

- **Difficulty / Target Companies:** `Easy | Amazon, Google, Bloomberg`
- **Problem Statement:** Given table Employee(id, name, salary, manager_id), find the names of employees who earn more than their direct manager.

Step-by-Step Walkthrough & Logic:

- Step 1: Join the Employee table with itself: alias `e` as Employee and `m` as Manager.
- Step 2: Join condition: `e.manager_id = m.id`.
- Step 3: Filter condition: `e.salary > m.salary`.
- Step 4: Project `e.name`.

Complete Solved Code / Calculation:

```
SELECT
    e.name AS Employee
FROM Employee e
INNER JOIN Employee m ON e.manager_id = m.id
WHERE e.salary > m.salary;
```

- **Line-by-Line Breakdown & Complexity Analysis:** Self join aligns each employee with their respective manager's record on the same row, enabling direct scalar comparison of salaries.

· 4. Practice Questions with Answers & Explanations

Practice Question 3.1

Question: If Table A has 5 rows and Table B has 10 rows, what is the minimum number of rows returned by a LEFT OUTER JOIN of A with B?

Options:

- A) 0
- B) 5
- C) 10
- D) 50

Answer: **B) 5** **Detailed Solution & Reasoning:** A LEFT JOIN preserves every row from the left table regardless of whether matches exist in the right table.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 3.1 (Oracle)

- **Interview Question:** _What is the difference between UNION and UNION ALL, and which should be preferred?_
- **Model High-Scoring Answer:** UNION combines the result sets of two queries and performs an explicit sorting/hashing pass to eliminate duplicate rows. UNION ALL combines the sets without deduplication. UNION ALL is vastly faster and consumes minimal memory. In production, always use UNION ALL unless duplicate elimination is explicitly required.
- **Interviewer Follow-Up & Pro Tip:** ? Mention the memory overhead and sorting cost associated with UNION.

CHAPTER 4

Subqueries, Correlated Subqueries & CTEs

Key Principle / Exam Takeaway: Master scalar subqueries, IN/EXISTS predicates, correlated subquery execution cost, and Common Table Expressions (WITH clause).

Core Curriculum Topics Covered in this Chapter:

- Scalar, column, and table-valued subqueries
- IN vs EXISTS: Performance in presence of NULLs
- Correlated subqueries: Row-by-row execution mechanics
- Common Table Expressions (CTEs) and readability benefits
- Recursive CTEs for organizational hierarchies and graph traversals

· 1. Core Concepts & In-Depth Explanation

A Subquery is a query nested inside another SQL statement. Subqueries can return single scalar values, lists of values, or virtual tables:

- **Non-Correlated Subquery:** Independent query evaluated once before the outer query runs.
- **Correlated Subquery:** References columns from the outer query row. The database engine must evaluate the subquery row-by-row for every candidate row of the outer query, which can lead to $O(N \times M)$ performance bottlenecks unless optimized into joins.
- **EXISTS vs IN:** `EXISTS` terminates as soon as a single matching row is found (short-circuit boolean evaluation) and handles `NULL` values gracefully. `NOT IN` returns empty if the subquery contains even a single `NULL`.
- **Common Table Expression (CTE):** Defined via `WITH cte_name AS (...)`, CTEs improve query modularity and readability, and support recursion for tree traversals.

· 2. Key Formulas, Rules & Complexity Classes

1. **NOT IN Null Trap:** $$\text{If } \text{Subquery returns } \{1, 2, \text{NULL}\}: \quad x \text{ NOT IN } \{\dots\} \implies \text{UNKNOWN} \implies \text{0 rows returned!}$$ $$\text{Always use: } \text{WHERE NOT EXISTS (\dots)} \quad \text{or filter } \text{IS NOT NULL}$$

· 3. Step-by-Step Solved Examples

Example 4.1: Find Second Highest Salary (CTE & Subquery)

- **Difficulty / Target Companies:** `Easy | Amazon, Microsoft, TCS`
- **Problem Statement:** Write an SQL query to report the second highest salary from the Employee table. If there is no second highest, return NULL.

Step-by-Step Walkthrough & Logic:

- Step 1: Distinct salaries eliminate duplicate values.
- Step 2: Filter out the absolute maximum salary.
- Step 3: Select the maximum of the remaining salaries.

Complete Solved Code / Calculation:

```
-- Approach 1: Subquery
SELECT MAX(salary) AS SecondHighestSalary
FROM Employee
WHERE salary < (SELECT MAX(salary) FROM Employee);

-- Approach 2: CTE with LIMIT/OFFSET
WITH RankedSalaries AS (
    SELECT DISTINCT salary
    FROM Employee
    ORDER BY salary DESC
    LIMIT 1 OFFSET 1
)
SELECT
    CASE
        WHEN COUNT(*) = 0 THEN NULL
        ELSE (SELECT salary FROM RankedSalaries)
    END AS SecondHighestSalary
FROM RankedSalaries;
```

- **Line-by-Line Breakdown & Complexity Analysis:** Approach 1 naturally returns NULL if no second distinct salary exists, without requiring outer wrappers.

· 4. Practice Questions with Answers & Explanations

Practice Question 4.1

Question: Which operator is safer when comparing against a subquery that might return NULL values?

Options:

- A) NOT IN
- B) NOT EXISTS
- C) !=
- D) <> ALL

Answer: **B) NOT EXISTS** **Detailed Solution & Reasoning:** NOT EXISTS evaluates boolean existence without evaluating scalar equality with NULL, preventing the three-valued logic failure.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 4.1 (Google)

- **Interview Question:** _How does a Recursive CTE work, and what is its standard termination condition?_

- **Model High-Scoring Answer:** A Recursive CTE contains two parts separated by UNION ALL: 1) The Anchor Member, which runs once to produce the base result set (e.g., finding the CEO at root level). 2) The Recursive Member, which references the CTE itself to join subsequent levels (e.g., finding employees whose manager is in the previous result set). Recursion halts automatically when the recursive member yields an empty set.

- **Interviewer Follow-Up & Pro Tip:** ? Mention the `MAXRECURSION` safeguard to prevent infinite loops.

CHAPTER 5

Advanced Window Functions & Analytical SQL

Key Principle / Exam Takeaway: Master OVER, PARTITION BY, ORDER BY, ROW_NUMBER, RANK, DENSE_RANK, NTILE, LAG, LEAD, and running cumulative aggregations.

Core Curriculum Topics Covered in this Chapter:

- Window Functions vs Group By: Preserving individual row identity
- Ranking functions: ROW_NUMBER(), RANK(), DENSE_RANK()
- Value navigation: LAG(), LEAD(), FIRST_VALUE(), LAST_VALUE()
- Frame specification: ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
- Cumulative running totals and moving averages

· 1. Core Concepts & In-Depth Explanation

Window Functions perform calculations across a set of table rows that are related to the current row, without collapsing rows like `GROUP BY` does. Each row retains its individual identity while gaining access to aggregated contextual metrics.

A window specification is defined by the `OVER()` clause:

- `PARTITION BY`: Divides the rows into partition groups (similar to GROUP BY).
- `ORDER BY`: Defines the logical sequence of rows within each partition.
- `ROWS / RANGE BETWEEN`: Defines the physical sliding frame of rows evaluated (e.g., moving averages).

Core ranking differences:

- `ROW_NUMBER()`: Assigns unique consecutive integers ($1, 2, 3, 4$). Ties receive arbitrary distinct numbers.
- `RANK()`: Assigns identical rank to ties, but skips subsequent numbers ($1, 2, 2, 4$).
- `DENSE_RANK()`: Assigns identical rank to ties without skipping numbers ($1, 2, 2, 3$).

· 2. Key Formulas, Rules & Complexity Classes

1. **Ranking Differences Summary:** $$\text{Values: } [100, 90, 90, 80]$$ $$\text{ROW_NUMBER: } [1, 2, 3, 4]$$ $$\text{RANK: } [1, 2, 2, 4]$$ $$\text{DENSE_RANK: } [1, 2, 2, 3]$$ 2. **Cumulative Running Total Invariant:** $$\text{SUM(amount) OVER (PARTITION BY account_id ORDER BY trans_date ROWS UNBOUNDED PRECEDING)}$$

· 3. Step-by-Step Solved Examples

Example 5.1: Top 3 Highest Paid Employees per Department (DENSE_RANK)

- **Difficulty / Target Companies:** `Medium | Amazon, Google, Goldman Sachs`
- **Problem Statement:** Find the employees who earn in the top 3 salaries for each department.

Step-by-Step Walkthrough & Logic:

- Step 1: Partition by `department_id` and order by `salary DESC` using `DENSE_RANK()`.
- Step 2: Wrap query in a CTE because window functions cannot appear directly in WHERE clauses.
- Step 3: Filter rows where `salary_rank <= 3`.

Complete Solved Code / Calculation:

```
WITH RankedEmployees AS (
    SELECT
        d.name AS Department,
        e.name AS Employee,
        e.salary AS Salary,
        DENSE_RANK() OVER (
            PARTITION BY e.department_id
            ORDER BY e.salary DESC
        ) AS rnk
    FROM Employee e
    JOIN Department d ON e.department_id = d.id
)
SELECT Department, Employee, Salary
FROM RankedEmployees
WHERE rnk <= 3
ORDER BY Department, Salary DESC;
```

- **Line-by-Line Breakdown & Complexity Analysis:** DENSE_RANK ensures that if two employees share the top salary, they both get rank 1 and the next distinct salary receives rank 2.

· 4. Practice Questions with Answers & Explanations

Practice Question 5.1

Question: Which window function retrieves a value from the immediately preceding row within the partition?

Options:

- A) LEAD()
- B) LAG()
- C) FIRST_VALUE()
- D) NTILE()

Answer: **B) LAG()** **Detailed Solution & Reasoning:** LAG(column, offset) accesses data from a previous row at a specified offset within the partition.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 5.1 (Amazon)

- **Interview Question:** _Can you use a Window Function in a WHERE clause? Why or why not?_
- **Model High-Scoring Answer:** No. In the SQL logical execution order, the WHERE clause evaluates before window functions (which execute alongside SELECT). Because the window calculations have not yet been performed when WHERE filters rows, referencing a window function in WHERE causes a syntax error. To filter by window values, wrap the query in a CTE or Subquery.

- **Interviewer Follow-Up & Pro Tip:** ? Cite the SQL logical execution sequence as the definitive reason.

CHAPTER 6

Database Indexing, Query Plans & Performance Optimization

Key Principle / Exam Takeaway: Master B+ Tree index mechanics, clustered vs non-clustered indexing, composite index leftmost prefix rule, and EXPLAIN ANALYZE interpretation.

Core Curriculum Topics Covered in this Chapter:

- B+ Tree index architecture: Height, fan-out, and leaf-level linked lists
- Clustered Index (physical ordering) vs Non-Clustered (Secondary) Index
- Composite Indexes and the Leftmost Prefix Rule
- Covering Indexes (Index-Only Scans) eliminating table lookups
- EXPLAIN and EXPLAIN ANALYZE: Sequential scan vs Index Scan vs Index Seek

· 1. Core Concepts & In-Depth Explanation

An Index is an auxiliary search data structure that enables fast data retrieval without scanning every row of a table. The universal standard for relational database indexes is the **B+ Tree**:

- All keys are stored in leaf nodes, which are linked together in a bidirectional doubly linked list, enabling rapid range scans.
- Internal nodes store only navigation guide keys and child pointers, achieving high fan-out (thousands of pointers per page) so that a tree of height 3 or 4 can index hundreds of millions of rows.

Clustered vs Non-Clustered Indexes:

- **Clustered Index:** Dictates the physical storage order of rows on disk. A table can have exactly ONE clustered index (usually the Primary Key). The leaf pages contain the actual data rows.
- **Non-Clustered Index:** Stored separately from data. The leaf pages contain index keys and row pointers (or the clustered key value) to locate the actual row.
- **Leftmost Prefix Rule:** For composite index `(A, B, C)`, queries filtering on `A`, or `A and B`, or `A and B and C` utilize the index. Queries filtering only on `B` or `C` cannot use the index.

· 2. Key Formulas, Rules & Complexity Classes

1. **B+ Tree Height & IO Cost:** $$\text{Height } h \approx \log_{\text{fanout}}(N) \implies \text{At most } 3-4 \text{ disk page I/Os for 100M rows}$$ 2. **Leftmost Prefix Validity:** $$\text{Index}(A, B, C): \quad \text{WHERE } A = 1 \implies \text{VALID}$$ $$\text{WHERE } A = 1 \text{ AND } B > 5 \implies \text{VALID}$$ $$\text{WHERE } B = 2 \implies \text{INVALID (Full Table Scan)}$$

· 3. Step-by-Step Solved Examples

Example 6.1: Optimizing a Slow Query with a Covering Composite Index

- **Difficulty / Target Companies:** `Medium | Uber, Amazon, TCS Digital`
- **Problem Statement:** The query `SELECT emp_id, salary FROM Employee WHERE dept_id = 5 AND is_active = TRUE ORDER BY salary DESC;` causes a full table scan on 5,000,000 rows. Design the optimal index.

Step-by-Step Walkthrough & Logic:

- Step 1: Identify equality filter columns: `dept_id` and `is_active`.
- Step 2: Identify sorting column: `salary`.
- Step 3: Identify projection columns: `emp_id` (assumed Primary Key).
- Step 4: Create composite index with equality columns first, followed by sort column: `(dept_id, is_active, salary DESC)`.

Complete Solved Code / Calculation:

```
-- Optimal Covering Index Creation
CREATE INDEX idx_emp_dept_active_sal
ON Employee (dept_id, is_active, salary DESC);

-- Verify execution plan changes from Seq Scan to Index Only Scan
EXPLAIN ANALYZE
SELECT emp_id, salary
FROM Employee
WHERE dept_id = 5 AND is_active = TRUE
ORDER BY salary DESC;
```

- **Line-by-Line Breakdown & Complexity Analysis:** Because all columns required by WHERE, ORDER BY, and SELECT reside inside the index leaf nodes, the engine executes an Index Only Scan without fetching base table pages.

· 4. Practice Questions with Answers & Explanations

Practice Question 6.1

Question: What happens if a query uses `WHERE UPPER(email) = 'TEST@EXAMPLE.COM'` when an index exists on column `email`?

Options:

- A) Fast index seek is performed
- B) The index is invalidated and a full table scan occurs unless a function-based index is created
- C) The database throws an error
- D) It automatically uses binary search

Answer: **B) The index is invalidated and a full table scan occurs unless a function-based index is created** **Detailed Solution & Reasoning:** Wrapping an indexed column inside a function prevents the engine from traversing the B+ Tree keys directly, causing a full table scan.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 6.1 (Google)

- **Interview Question:** _Why do B+ Trees use wide fan-out (thousands of children per node) instead of binary search trees for disk storage?_
- **Model High-Scoring Answer:** Disk I/O is orders of magnitude slower than CPU memory access. Binary trees have a fan-out of 2, requiring height $O(\log_2 N)$ (approx 30 disk reads for 1 billion rows). B+ Trees match node sizes to physical disk page blocks (e.g., 4KB or 8KB), achieving fan-out of 500 to 2000. This compresses tree height to 3 or 4 levels, guaranteeing that any record can be fetched in at most 3 to 4 disk I/O operations.

- **Interviewer Follow-Up & Pro Tip:** ? Connect node capacity to physical disk block page size.

CHAPTER 7

Transactions, ACID Properties & Concurrency Isolation

Key Principle / Exam Takeaway: Master Atomicity, Consistency, Isolation, Durability, the four ANSI SQL isolation levels, and concurrency anomalies (Dirty Read, Non-Repeatable Read, Phantom Read).

Core Curriculum Topics Covered in this Chapter:

- ACID properties and WAL (Write-Ahead Logging) durability mechanics
- Concurrency anomalies: Dirty Reads, Non-Repeatable Reads, Phantom Reads
- ANSI SQL Isolation Levels: READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE
- Multi-Version Concurrency Control (MVCC) in PostgreSQL and MySQL InnoDB
- Pessimistic locking (SELECT FOR UPDATE) vs Optimistic locking (version columns)

· 1. Core Concepts & In-Depth Explanation

A Transaction is a logical unit of work encompassing one or more SQL statements executed against a database. Transactions must adhere to the ACID principles:

- **Atomicity:** All-or-nothing execution. If any step fails, the entire transaction rolls back via Write-Ahead Logging (WAL) undo logs.
- **Consistency:** Transactions transition the database from one valid state satisfying all schema constraints to another.
- **Isolation:** Concurrent transactions execute without interfering with one another.
- **Durability:** Committed transactions persist permanently, surviving hardware crashes or power outages via redo logs.

Concurrency Anomalies: 1. **Dirty Read:** Reading uncommitted changes made by another concurrent transaction that subsequently rolls back. 2. **Non-Repeatable Read:** Re-reading the same row returns different column values because another transaction modified and committed it in between. 3. **Phantom Read:** Re-executing a range query returns a different number of rows because another transaction inserted or deleted rows matching the range.

· 2. Key Formulas, Rules & Complexity Classes

1. **Isolation Levels vs Anomalies:**

- **READ UNCOMMITTED:** Dirty Read (YES), Non-Repeatable Read (YES), Phantom Read (YES)
- **READ COMMITTED:** Dirty Read (NO), Non-Repeatable Read (YES), Phantom Read (YES)
- **REPEATABLE READ:** Dirty Read (NO), Non-Repeatable Read (NO), Phantom Read (YES/Engine dependent)
- **SERIALIZABLE:** Dirty Read (NO), Non-Repeatable Read (NO), Phantom Read (NO)

· 3. Step-by-Step Solved Examples

Example 7.1: Preventing Lost Updates with Optimistic Locking

- **Difficulty / Target Companies:** `Medium | Amazon, Flipkart, Goldman Sachs`
- **Problem Statement:** Two concurrent users attempt to book the final ticket on a flight. Prevent race conditions and double booking.

Step-by-Step Walkthrough & Logic:

- Step 1: Include a `version` integer column in the `Flight` table.
- Step 2: Read current `available_seats` and current `version`.
- Step 3: Update only if `version` has not changed: `UPDATE Flight SET available_seats = available_seats - 1, version = version + 1 WHERE flight_id = 101 AND version = @read_version`.
- Step 4: Check affected rows: if 0, another transaction updated first; rollback and retry.

Complete Solved Code / Calculation:

```
-- Optimistic Locking Execution
-- User 1 reads: flight_id = 101, seats = 1, version = 4
UPDATE Flight
SET available_seats = available_seats - 1,
    version = version + 1
WHERE flight_id = 101
  AND version = 4
  AND available_seats > 0;

-- In application: if affected_rows == 0, abort booking and alert user
```

- **Line-by-Line Breakdown & Complexity Analysis:** Optimistic locking prevents long database row locks. Only the transaction that commits first succeeds; the second receives 0 affected rows and safely retries.

· 4. Practice Questions with Answers & Explanations

Practice Question 7.1

Question: Which concurrency anomaly is prevented by moving from READ COMMITTED to REPEATABLE READ?

Options:

- A) Dirty Read
- B) Non-Repeatable Read
- C) Deadlock
- D) Division by zero

Answer: **B) Non-Repeatable Read** **Detailed Solution & Reasoning:** REPEATABLE READ ensures that any data read during a transaction remains identical on subsequent reads throughout that transaction.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 7.1 (Amazon)

- **Interview Question:** _How does Multi-Version Concurrency Control (MVCC) eliminate read-write blocking?_

- **Model High-Scoring Answer:** In traditional locking systems, readers acquire shared locks that block writers, and writers acquire exclusive locks that block readers. MVCC allows 'readers to not block writers, and writers to not block readers'. When a row is updated, the engine does not overwrite it in-place; instead, it writes a new version of the row with a transaction creation timestamp (`xmin`). A reading transaction reads a consistent snapshot of data corresponding to its own start time, completely isolated from concurrent in-flight updates.

- **Interviewer Follow-Up & Pro Tip:** ? Mention snapshot isolation and visibility checks based on transaction IDs.

CHAPTER 8

Solved Placement SQL Queries & Technical Coding Challenges

Key Principle / Exam Takeaway: Master top placement SQL coding rounds: consecutive active days, department salary thresholds, user retention churn, and cancellation rates.

Core Curriculum Topics Covered in this Chapter:

- Consecutive numbers / active days using LAG and ROW_NUMBER difference technique
- Trips and Users cancellation rate calculation
- Department top salary and cumulative department payroll
- Monthly active users (MAU) and month-over-month user retention rate

· 1. Core Concepts & In-Depth Explanation

This capstone chapter covers advanced SQL coding scenarios asked in campus recruitment rounds at FAANG, Uber, Stripe, and financial investment banks.

Mastering two foundational SQL patterns solves the majority of complex interview queries: 1. **Gaps and Islands (Consecutive Days / Sessions):** To identify sequences of consecutive active days, compute `day - ROW_NUMBER() OVER (ORDER BY day)`. For any continuous unbroken island, the difference between the calendar date and the sequence index produces an identical constant date! Grouping by this constant identifies all contiguous streaks. 2. **Conditional Aggregation with FILTER / CASE WHEN:** Calculating dynamic rates (such as cancellation rate or churn percentage) in a single pass: `ROUND(SUM(CASE WHEN status = 'cancelled' THEN 1 ELSE 0 END) * 1.0 / COUNT(*), 2)`.

· 2. Key Formulas, Rules & Complexity Classes

1. **Gaps and Islands Date Difference Formula:** $$\text{IslandID} = \text{date} - \text{ROW_NUMBER}() \times \text{INTERVAL '1 day'}$$ $$\text{All contiguous dates produce identical } \text{IslandID}!$$ 2. **Cancellation Rate Formula:** $$\text{Rate} = \frac{\sum(\text{Cancelled Trips})}{\text{Total Valid Trips}}$$

· 3. Step-by-Step Solved Examples

Example 8.1: Consecutive Numbers (3 or More in a Row)

- **Difficulty / Target Companies:** `Medium | Amazon, LeetCode 180`
- **Problem Statement:** Find all numbers that appear at least three times consecutively in a table `Logs(id, num)` where id is an auto-incrementing integer.

Step-by-Step Walkthrough & Logic:

- Step 1: Use `LAG()` and `LEAD()` to inspect immediate predecessor and successor values for every row.
- Step 2: Check if current `num` equals both preceding `num` and succeeding `num`.
- Step 3: Select distinct values to avoid duplicate outputs.

Complete Solved Code / Calculation:

```
WITH InspectedLogs AS (
    SELECT
        num,
        LAG(num, 1) OVER (ORDER BY id) AS prev_num,
        LEAD(num, 1) OVER (ORDER BY id) AS next_num
    FROM Logs
)
SELECT DISTINCT num AS ConsecutiveNums
FROM InspectedLogs
WHERE num = prev_num AND num = next_num;
```

- **Line-by-Line Breakdown & Complexity Analysis:** LAG and LEAD allow single-pass comparison across neighboring rows in $O(N)$ time, avoiding self-joins on N^3 combinations.

Example 8.2: Trips and Users (Cancellation Rate of Unbanned Users)

- **Difficulty / Target Companies:** `Hard | Uber, Google`
- **Problem Statement:** Write an SQL query to find the cancellation rate of requests with unbanned users (both client and driver must not be banned) each day between '2026-10-01' and '2026-10-03'.

Step-by-Step Walkthrough & Logic:

- Step 1: Join Trips with Users table twice: once for client, once for driver.
- Step 2: Filter out any trip where either client or driver is banned (`banned = 'Yes'`).
- Step 3: Group by trip `request_at` date.
- Step 4: Compute cancellation percentage using conditional SUM.

Complete Solved Code / Calculation:

```
SELECT
    t.request_at AS Day,
    ROUND(
        SUM(CASE WHEN t.status IN ('cancelled_by_driver', 'cancelled_by_client') THEN 1.0 ELSE
        COUNT(*),
        2
    ) AS "Cancellation Rate"
FROM Trips t
JOIN Users c ON t.client_id = c.users_id AND c.banned = 'No'
JOIN Users d ON t.driver_id = d.users_id AND d.banned = 'No'
WHERE t.request_at BETWEEN '2026-10-01' AND '2026-10-03'
GROUP BY t.request_at
ORDER BY t.request_at;
```

- **Line-by-Line Breakdown & Complexity Analysis:** Filter conditions on the JOIN clauses proactively drop banned users before aggregation. The CASE statement calculates the fractional cancellation rate rounded to 2 decimal places.

· 4. Practice Questions with Answers & Explanations

Practice Question 8.1

Question: In the Gaps and Islands problem, what does grouping by `date - ROW_NUMBER()` achieve?

Options:

- A) It sorts the table in descending order

- B) It groups contiguous unbroken daily streaks into identical key groups
- C) It deletes duplicate rows
- D) It pivots columns into rows

Answer: **B) It groups contiguous unbroken daily streaks into identical key groups** **Detailed Solution & Reasoning:** For contiguous dates, the date increments by 1 day as the row number increments by 1, making their difference a constant value representing the island.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 8.1 (Uber)

- **Interview Question:** _How would you find monthly active users (MAU) who were also active in the immediately preceding month (Month-over-Month Retention)?_
- **Model High-Scoring Answer:** Extract distinct `user_id` and truncated month: `DATE_TRUNC('month', activity_date) AS active_month`. Self-join this CTE on `curr.user_id = prev.user_id AND curr.active_month = prev.active_month + INTERVAL '1 month'`. Group by `curr.active_month` and compute retained user count divided by total active users in `prev.active_month`.

- **Interviewer Follow-Up & Pro Tip:** ? Clearly describe the self-join interval match.

· Campus Placement Certification & Final Exam Policy

To earn Placement-Ready Master status, students must complete all chapter practice questions, review all solved examples, and score at least 80% on the comprehensive subject assessment.